RESILIENCE ENGINEERING

DESIGN FOR FAILURE

The question is not whether components will fail — it is when. Design every system assuming any component can fail at any time. Graceful degradation, circuit breakers, bulkheads, retry logic with exponential backoff, and timeout policies bound failure and make it recoverable.

KEY BENEFITS

- ✓ Converts total outages into degraded-but-functional states users can tolerate
- ✓ Circuit breakers prevent a struggling service from triggering a cascade
- ✓ Bulkheads contain failure zones — one bad actor cannot consume all resources
- ✓ Explicit failure handling makes on-call response predictable and procedural

REAL-WORLD EXAMPLE

AWS's cell-based architecture isolates failures to a defined blast radius. When one availability zone fails, it cannot propagate to adjacent cells — preventing numerous total regional outages from a single component failure.

PRACTICAL EXAMPLE

The recommendation engine goes down. Rather than an error, the product page falls back to a static 'popular items' cache. Conversion rate drops 4%. When recommendations recover, the fallback deactivates automatically. No incident declared.

REFLECTION QUESTION

For each critical dependency, can you describe exactly what your system does when that dependency becomes unavailable?